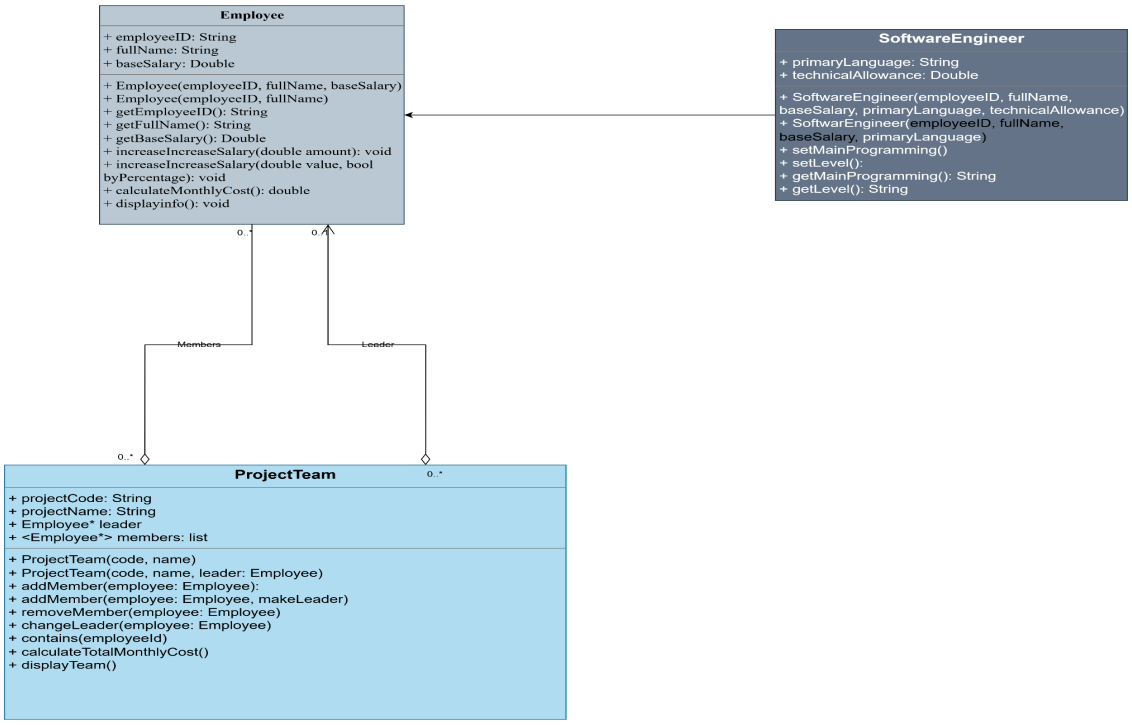
1. **The design diagram**



2. **Link GitHub của bài 3**
   https://github.com/dnb1010/OOP-Assignment/tree/main/B%C3%A0i%203

3. **Test**

```
PS C:\Users\admin\Documents\OOP> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\admin\Documents\OOP\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\admin\Documents\OOP> & c:\Users\admin\Documents\OOP\.venv\Scripts\python.exe "c:/Users/admin/Documents/OOP/Bài 3/main.py"
Thêm lại E01: False

Dự án: P01 - Hệ thống quản lý nhân sự
Trưởng nhóm: Minh (S02)
Danh sách nhân sự:
Nhân viên: E01 | Bảo | 1000000.0
Nhân viên: E02 | An | 11000000.0
Kỹ sư phần mềm: S01 | Linh| Ngôn ngữ: C++| Lương cơ bản: 0.0| Phụ cấp: 0.0| Chi phí: 0
Kỹ sư phần mềm: S02 | Minh| Ngôn ngữ: Python| Lương cơ bản: 15000000.0| Phụ cấp: 2000000.0| Chi phí: 17,000,000
Tổng chi phí hằng tháng: 29,000,000

Xóa trưởng nhóm S02: False

Dự án: P01 - Hệ thống quản lý nhân sự
Trưởng nhóm: An (E02)
Danh sách nhân sự:
Nhân viên: E01 | Bảo | 1000000.0
Nhân viên: E02 | An | 11000000.0
Kỹ sư phần mềm: S01 | Linh| Ngôn ngữ: C++| Lương cơ bản: 0.0| Phụ cấp: 0.0| Chi phí: 0
Tổng chi phí hằng tháng: 12,000,000

Dự án: P02 - Dự án AI
Trưởng nhóm: Bảo (E01)
Danh sách nhân sự:
Nhân viên: E01 | Bảo | 1000000.0
Tổng chi phí hằng tháng: 1,000,000

Kết thúc phạm vi của nhóm thứ hai.
[Hủy ProjectTeam] P02

E01 vẫn tồn tại:
Nhân viên: E01 | Bảo | 1000000.0
[Đúng: Mã rỗng] Mã nhân sự không được rỗng.
[Hủy Employee] ?
[Đúng: Tên rỗng] Họ và tên không được rỗng.
[Hủy Employee] X01
[Đúng: Lương âm] Lương cơ bản không được âm.
[Hủy Employee] X02
[Đúng: Mức tăng bằng 0] Giá trị tăng lương phải lớn hơn 0.
[Đúng: Mức tăng âm] Giá trị tăng lương phải lớn hơn 0.
[Đúng: Ngôn ngữ rỗng] Ngôn ngữ chính không được rỗng.
[Hủy SoftwareEngineer] X03
[Hủy Employee] X03
[Đúng: Phụ cấp âm] Phụ cấp kĩ thuật không được âm.
[Hủy SoftwareEngineer] X04
[Hủy Employee] X04

Tất cả kiểm thử đã vượt qua.
[Hủy ProjectTeam] P03
[Hủy ProjectTeam] P01
[Hủy Employee] E01
[Hủy Employee] E01
[Hủy SoftwareEngineer] S02
```

```python
    # 1. Hai Employee bằng hai cách khởi tạo khác nhau.
    e1 = Employee("E01", "Bảo")
    e2 = Employee("E02", "An", 10_000_000)

    # 2. Hai SoftwareEngineer bằng hai cách khởi tạo.
    s1 = SoftwareEngineer("S01", "Linh", "C++")

    s2 = SoftwareEngineer(
        "S02", "Minh", "Python",
        baseSalary=15_000_000,
        technicalAllowance=2_000_000
    )

    # 3. Tăng lương cố định.
    e1.increaseSalary(1_000_000)
    assert e1.baseSalary == 1_000_000

    # 4. Tăng lương theo phần trăm.
    e2.increaseSalary(10, True)
    assert math.isclose(e2.baseSalary, 11_000_000)

    # 5. Tạo nhóm chưa có trưởng nhóm.
    team1 = ProjectTeam("P01", "Hệ thống quản lý nhân sự")
    assert team1.leader is None

    # 6. Thêm nhân sự.
    assert team1.addMember(e1)
    assert team1.addMember(e2)
    assert team1.addMember(s1)

    # 7. Thêm kỹ sư và đặt làm trưởng nhóm.
    assert team1.addMember(s2, True)
    assert team1.leader is s2

    # 8. Thử thêm trùng.
    result = team1.addMember(e1)
    print("Thêm lại E01:", result)
    assert result is False
    assert len(team1.members) == 4

    # 9. Hiển thị bằng lời gọi đa hình.
    team1.displayTeam()

    # 10. Tính tổng chi phí.
    assert math.isclose(
        team1.calculateTotalMonthlyCost(),
        29_000_000
    )
```

```python
# 11. Không được xóa trưởng nhóm.
result = team1.removeMember("S02")
print("\nXóa trưởng nhóm S02:", result)
assert result is False

# 12. Đổi trưởng nhóm rồi xóa trưởng nhóm cũ.
assert team1.changeLeader(e2)
assert team1.contains("S02")  # Người cũ vẫn còn trong nhóm.
assert team1.removeMember("S02")

assert team1.leader is e2
assert math.isclose(
    team1.calculateTotalMonthlyCost(),
    12_000_000
)

team1.displayTeam()

# 13-14. Python không có phạm vi biến riêng cho khối if/for.
# Dùng hàm để tạo phạm vi cục bộ cho nhóm thứ hai.
def second_team_demo():
    team2 = ProjectTeam("P02", "Dự án AI", e1)

    assert team1.contains("E01")
    assert team2.contains("E01")
    assert team2.leader is e1

    team2.displayTeam()
    print("\nKết thúc phạm vi của nhóm thứ hai.")

second_team_demo()

# 15. Nhân sự vẫn tồn tại sau khi nhóm thứ hai hết phạm vi.
print("\nE01 vẫn tồn tại:")
e1.displayInfo()
assert team1.contains("E01")

# Kiểm thử các trường hợp biên.
empty_team = ProjectTeam("P03", "Nhóm rỗng")
assert empty_team.calculateTotalMonthlyCost() == 0
assert empty_team.removeMember("NOT_FOUND") is False

# Người chưa thuộc nhóm được tự thêm khi đổi trưởng nhóm.
assert empty_team.changeLeader(s1)
assert empty_team.contains("S01")
assert empty_team.leader is s1

# Đối tượng khác nhưng cùng mã bị từ chối.
duplicate = Employee("E01", "Người khác")
assert team1.addMember(duplicate) is False
assert team1.changeLeader(duplicate) is False
```

```
    # Kiểm thử các trường hợp biên.
    empty_team = ProjectTeam("P03", "Nhóm rỗng")
    assert empty_team.calculateTotalMonthlyCost() == 0
    assert empty_team.removeMember("NOT_FOUND") is False

    # Người chưa thuộc nhóm được tự thêm khi đổi trưởng nhóm.
    assert empty_team.changeLeader(s1)
    assert empty_team.contains("S01")
    assert empty_team.leader is s1

    # Đối tượng khác nhưng cùng mã bị từ chối.
    duplicate = Employee("E01", "Người khác")
    assert team1.addMember(duplicate) is False
    assert team1.changeLeader(duplicate) is False

    invalid_cases = [
        ("Mã rỗng", lambda: Employee("", "A")),
        ("Tên rỗng", lambda: Employee("X01", " ")),
        ("Lương âm", lambda: Employee("X02", "A", -1)),
        ("Mức tăng bằng 0", lambda: e1.increaseSalary(0)),
        ("Mức tăng âm", lambda: e1.increaseSalary(-100)),
        (
            "Ngôn ngữ rỗng",
            lambda: SoftwareEngineer("X03", "A", "")
        ),
        (
            "Phụ cấp âm",
            lambda: SoftwareEngineer(
                "X04", "A", "Python", technicalAllowance=-1
            )
        ),
    ]

    for label, operation in invalid_cases:
        try:
            operation()
        except ValueError as error:
            print(f"[Đúng: {label}] {error}")
        else:
            raise AssertionError(f"Chưa chặn trường hợp: {label}")

    print("\nTất cả kiểm thử đã vượt qua.")

    # Bỏ các nhóm trước khi kết thúc phạm vi chứa nhân sự.
    del empty_team
    del team1
```